Post-Transformer Architectures: Fixed Memory vs. Growing Context

Can a tiny memory remember an infinite stream?

> *"A fixed-size recurrent state can process arbitrarily long sequences without allocating a new memory slot for each token, but storing more associations in that fixed state can cause retrieval interference."*

› Model Assumptions & Limitations (Click to view)

1. Make a Prediction

We have an 8x8 state matrix. We will store 20 facts with 80% shared-key component. What do you expect will happen to recall?

- ○ Select an option...
- ◉ Recall will increase
- ○ Recall will decrease
- ○ Recall will stay similar
- ○ The system runs out of GPU memory

---

2. Write Information into Fixed State

🟢 **LIVE EXPERIMENT** | Generated and evaluated now in PyTorch.

What is stored?

There is no new row called 'Paris'. The association is distributed across the existing state.

Key k
'Capital of France'

	0
0	-(
1	0
2	-(
3	-(
4	0
5	-(
6	0
7	-(

Value v
'Paris'

	0
0	-(
1	0
2	-(
3	-(
4	-(
5	-(
6	-(
7	-0.931603

Outer Product vk^T

	0	1	2	3
0	0.006632	-0.287580	0.259148	(
1	-0.008675	0.376184	-0.338992	-(
2	0.016458	-0.713686	0.643127	(
3	0.014113	-0.611971	0.551467	(
4	0.002415	-0.104715	0.094362	(
5	0.005108	-0.221512	0.199611	(
6	0.025038	-1.085718	0.978377	(
7	0.009424	-0.408643	0.368242	(

State Matrix M (After Addition)

	0	1	2	3
0	0.006632	-0.287580	0.259148	0.221927
1	-0.008675	0.376184	-0.338992	-0.290303
2	0.016458	-0.713686	0.643127	0.550754
3	0.014113	-0.611971	0.551467	0.472260
4	0.002415	-0.104715	0.094362	0.080809
5	0.005108	-0.221512	0.199611	0.170941
6	0.025038	-1.085718	0.978377	0.837852
7	0.009424	-0.408643	0.368242	0.315351

🟣 **Frontier Connection: Dragon Hatchling (BDH)**

Our toy model updates a dense matrix M via outer-product writes. In contrast, **BDH** achieves this $O(1)$ state through a scale-free network of locally interacting neurons, utilizing **evolving synaptic/edge state** as working memory. The BDH paper reports that this synaptic state carries interpretable information—for instance, concept-related inputs dynamically lead to specific synaptic responses (e.g., tracking currencies).

---

3. Retrieve Information (Truth vs Prediction)

Actual Result: Recall = 10.0%

Your prediction was: **Recall will increase**

Retrieval Log (First 5):

Query: Capital of France | **Truth:** Paris | **Output:** Mars ❌ Interference

Query: Largest planet | **Truth:** Jupiter | **Output:** Mars ❌ Interference

Query: Author of Hamlet | **Truth:** Shakespeare | **Output:** Mars ❌ Interference

Query: Freezing point of water | **Truth:** 0°C | **Output:** Mars ❌ Interference

Query: Red planet | **Truth:** Mars | **Output:** Mars ✅ Correct

---

4. Increase Facts / Observe Interference

⚪ **FORMAL** | State shape doesn't depend on sequence length.

Facts	Mean Recall (%)	Std Dev (%)	State Size
5	48.0000	20.3961	8x8
20	9.0000	4.8990	8x8
50	3.2000	0.9798	8x8

Why does it happen?

> Every new association is added to the same state. When keys overlap, their writes overlap too. A query therefore receives contributions from multiple stored associations.

Fact A: $key_A \times value_A \rightarrow \text{SAME STATE}$ **Fact B:** $key_B \times value_B \rightarrow \text{SAME STATE}$

If $key_A \approx key_B$, then $write_A \approx write_B$. Retrieval becomes mixed.

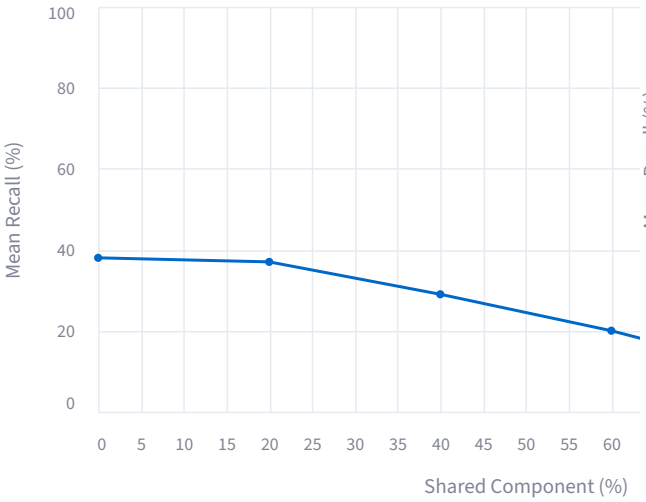
🟣 **Frontier Connection: BDH-CQ & Capacity Limits**
BDH-CQ uses a recurrent contextual memory to store history: $S_t = U_\epsilon(S_{t-1}, D_t)$, where demonstrations D_t iteratively update the memory. It then performs latent reasoning on this compressed state: $H_{r+1} = F_\epsilon(H_r, S_K)$. While much richer than our dense matrix, even advanced recurrent states face capacity and interference limits. The BDH-CQ technical report notes that it still struggles with certain transformations, reaching only 2/10 on 'Copy and Order' tasks when representation changes or compositionality is required.

representation changes or compositionality is required.

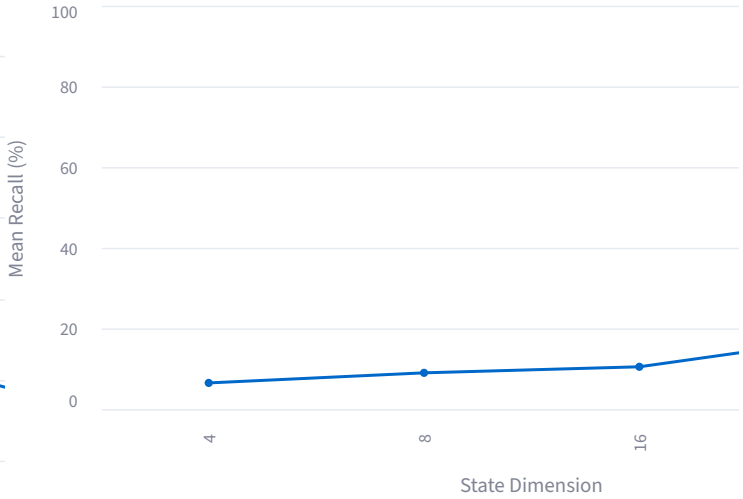
---

6. The Systemic Trade-Off

Recall vs Shared-Key Component



Recall vs State Size



---

7. Compare Architecture (KV Cache vs Fixed State)

🔵 **ANALYTICAL** | Calculated from the KV-cache formula.

KV Cache

✓ precise history
✗ memory grows with context

Fixed State

✓ bounded state
✗ information must be compressed
✗ interference can occur

Under our assumptions (12 layers, 8 heads, 64-dim, FP16), the K/V cache footprint grows linearly with sequence length:

Sequence Length	Memory (GB)
1000	0.0229
10000	0.2289
100000	2.2888
1000000	22.8882

This calculation covers the K/V tensors only, not model weights, activations, or allocator overhead.

---

8. 60 Second Test

8. 60-Second Test

1. If the number of facts increases while state dimension stays fixed, what happens?

- Select...
- State grows
- Recall may degrade
- KV cache appears
- Nothing changes

2. Why?

- Select...
- The state gets physically larger
- Multiple associations are superposed into the same state, creating interference
- The model stops computing

3. What happens to KV-cache memory as sequence length grows?

- Select...
- Remains fixed
- Grows linearly under stated assumptions

4. What makes BDH different from our toy?

- Select...
- Same as our matrix
- BDH uses a richer network of neuron-like units with evolving synaptic/edge state

Score Challenge

---

9. Explain it yourself

Why can a fixed-size state use constant memory while still suffering retrieval interference?

Reveal Model Answer

---

10. Free Sandbox: Break the memory yourself

State Dimension: 4

Facts: 20

Shared-Key Component (%): 80

Seed: 42

(The experiment runs automatically when you change the sliders)

State size: 4x4 | **Facts stored:** 20 | **Measured correlation:** 0.64

Recall: 5.0%